Last changes : 10.05.21, 17:33

# Task 3 - Summary, Questions, Pseudo-Codes

## Steps

**1.** Make a summary of the linked elements.

We have the following elements:

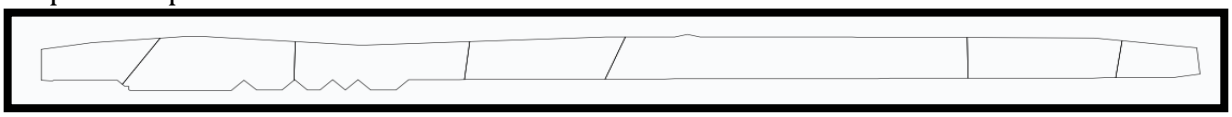
A. IDs of the elements + characters in the "/ground-truth/transcription.txt" file.

```
3    270-01-03 O-r-d-e-r-s
```

We build the ground truth by reading the "transcription.txt" file and then generating a 2D list where the first dimension is 14 (pages) and the second dimension is the list of words in that specific page.

B. Shapes of the elements (graphics + textual) + IDs of the elements in the "/ground-truth/locations/xxx.svg" files where xxx is the number of the image.

Graphical representation:

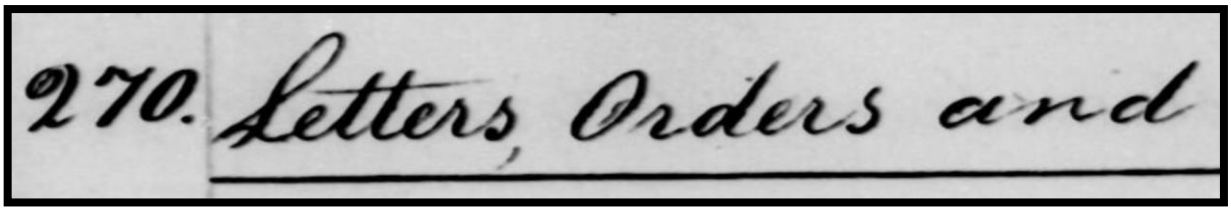


Textual representation (of one shape):

```
<path fill="none" d="M 512.00 229.00 L 532.00 249.00 L 552.00 249.00
L 511.00 230.00 Z" stroke-width="1" id="270-01-03" stroke="black"/>
```

...

C. JPG images in the "/images/xxx.jpg" files where xxx is the number of the image.




We can then link A to B with the IDs (here 270-01-03), B to C with the coordinates and A to C by the link A -> B -> C.

At the end, the result of the summary will be as follow:

| IDs | Characters | Position | Binary Image (For point 2) |
|---|---|---|---|

| | | | |
|---|---|---|---|
| … | … | … | … |
| 270-01-03 | O-r-d-e-r-s | [M 512:00 229.00 L 532.00 249.00 …] | … |
| … | … | … | … |

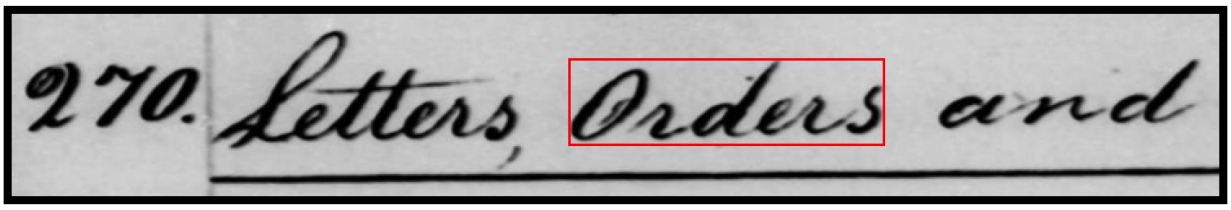
To capture SVG content, we could use regular expressions.
Then, the pseudo-code would be:

```
Import regular expressions
transcriptions = "/ground-truth/transcription.txt"
summary = create 2D table
for i in range (0,len("transcription until ID number 300-02-01")):
        summary[0][i] = "regular expression to capture the id in transcription, line i"
        summary[1][i] = "regular expression to capture the characters in transcription,
        line i"
        id = "regular expression to capture the image number in the ID in summary[0][i]"
        shape = "/ground-truth/locations/"+id+".svg"
        summary[2][i] = "regular expression to capture the position in shape equals to
summary[0][i]"
```

**2.** Crop and binarize the image.

For this part, we made the choice to crop rectangle boxes for the moment.
The binarization function is implemented using Python cv2 libraries.

For one element, to crop the image, we must again use regular expression.
It will capture each coordinate in the Summary[2][i] position.
Then, we will only keep the min. x and y and the max. x and y to have the border of the rectangle. With this operation, we are sure that the element is all inside the rectangle.




Then, we will apply the binarization function on the image and store the result in summary[3][i].

Then, the pseudo-code would be:

```
for i in range (0, len(summary))
        minX = "maximal value of position x for an A4 page"
        maxX = 0
        minY = "maximal value of position y for an A4 page"
        maxY = 0
        while ("regular expression to capture the next coordinate in summary[2] in line i"
        != empty):
                if (minX > "x coordinate in the expression"):
                        minX = "x coordinate in the expression"
```

```
            if (maxX < "x coordinate in the expression"):
                maxX = "x coordinate in the expression"
            if (minY > "y coordinate in the expression"):
                minY = "y coordinate in the expression"
            if (maxY > "y coordinate in the expression"):
                maxY = "y coordinate in the expression"
        rectangleCoordinates = [minX, maxX, minY, maxY]
        image = "/images/"+summary[0][i]+".jpg"
        summary [3][i] = "crop rectangle in image to have it into binary numbers"
```

3. Use the DTW algorithm (with Sakoe-Chiba to be faster) to calculate distances. We must find (the word we want to find in the list in the "/task/keywords.txt" file) in our summary (maybe there are many times this word) and then compare the word(s) in the summary with all the words in the validation set to find which one(s) are the one researched.

We can use the features functions (listed below) as an input to calculate the distances.

1. Lower contour
2. Upper contour
3. Transition from balck pixel to white
4. Fraction of total balck pixels in the window
5. Fraction of black pixels between LC and UC
6. Gradient of feature 5

4. We must calculate precision and recall for the whole training set and also calculate it with the validating set.

The training set consists of the 10 images from number 270 to 279 and the validation set consists of the 5 images from number 300 to 304.

We have the following elements:

TP = We search the word "example", we find that a word x is "example" in the validation set and it is really written "example".
FP = We search the word "example", we find that a word x is "example" in the validation set but it is NOT written "example", it is written "school".
FN = We search the word "example", we find that a word x is NOT "example" in the validation set, but it was written "example".
TN = We search the word "example", we find that a word x is NOT "example" in the validation set, and it was really not written "example", it was written "school".

Then, Precision is $\frac{TP}{TP+FP}$ and Recall is $\frac{TP}{TP+FN}$.